目录

1 Bagging **5**

1.1 核心定义 5

1.2 核心原理：Bootstrap Sampling 5

1.2.1 什么是自助采样? 5

1.2.2 包外估计 (Out-of-Bag, OOB) 5

1.3 数学原理：为什么能降低方差? 6

1.4 典型代表：随机森林 (Random Forest) 6

1.5 Bagging vs. Boosting 6

1.6 关键代码片段 7

1.7 深度面试题 7

1.8 参考文献与拓展 8

2 Boosting **9**

2.1 核心定义 9

2.2 加法模型与前向分布算法 9

2.3 典型算法 1：AdaBoost (Adaptive Boosting) 10

2.3.1 核心机制 10

2.3.2 损失函数视角 10

2.4 典型代表 2：GBDT (Gradient Boosting Decision Tree) 10

2.4.1 核心思想：拟合负梯度 10

2.5 典型代表 3：XGBoost (Extreme Gradient Boosting) 10

2.5.1 目标函数的严谨推导 11

2.5.2 分裂收益 (Gain) 公式 11

2.6 典型代表 4：LightGBM (Light Gradient Boosting Machine) 11

2.6.1 GOSS (基于梯度的单侧采样) 11

2.6.2 EFB (互斥特征捆绑) 12

2.6.3 Leaf-wise vs Level-wise 12

2.7 Boosting 家族综合对比 12

2.8 关键代码片段 12

2.9 深度面试题 13
2.10 参考文献 13

3 Decision_tree 15
3.1 核心定义 15
3.2 核心原理：如何选择最优划分特征? 15
3.2.1 信息增益 (Information Gain) —ID3 15
3.2.2 信息增益率 (Gain Ratio) —C4.5 16
3.2.3 基尼指数 (Gini Index) —CART 16
3.3 三大经典算法对比 17
3.4 决策树示意图 17
3.5 剪枝 (Pruning) 17
3.5.1 预剪枝 (Pre-pruning) 17
3.5.2 后剪枝 (Post-pruning) 18
3.6 CART 回归树 18
3.7 集成学习扩展 18
3.8 关键代码片段 18
3.9 深度面试题 19
3.10 参考文献与拓展 19

4 Lda 21
4.1 核心定义 21
4.2 数学推导：Fisher 判别准则 21
4.2.1 类内散度矩阵 (Within-class Scatter Matrix) 21
4.2.2 类间散度矩阵 (Between-class Scatter Matrix) 21
4.2.3 最优化目标 22
4.2.4 解的求取 22
4.3 多分类扩展 22
4.4 LDA vs. PCA 22
4.5 局限性 22
4.6 关键代码片段 23
4.7 深度面试题 23
4.8 参考文献与拓展 24

5 Logistic_regression 25
5.1 核心定义 25
5.2 数学建模：从线性回归到逻辑回归 25
5.2.1 为什么不能直接用线性回归做分类? 25

5.2.2 对数几率视角：Sigmoid 从何而来? 25
5.2.3 Sigmoid 函数的关键性质 26
5.2.4 概率模型 27
5.3 损失函数推导：为什么用交叉熵? 27
5.3.1 极大似然估计 (MLE) 27
5.3.2 梯度推导 27
5.4 决策边界 28
5.5 正则化 28
5.6 多分类扩展：Softmax 回归 28
5.7 关键代码片段 29
5.8 逻辑回归 vs 其他模型 29
5.9 深度面试题 29
5.10 参考文献与拓展 30

6 Pca **31**
6.1 核心定义 31
6.2 数学推导：最大方差视角 31
6.2.1 问题建模 31
6.2.2 拉格朗日乘子法求解 32
6.3 另一种视角：最小重构误差 32
6.4 算法步骤 32
6.5 SVD 实现的优势 32
6.6 PCA vs. LDA 33
6.7 关键代码片段 33
6.8 深度面试题 33
6.9 参考文献 34

7 Svm **35**
7.1 核心定义 35
7.2 线性可分 SVM：硬间隔 35
7.2.1 几何直觉：为什么要最大化间隔? 35
7.2.2 数学建模 35
7.2.3 最大间隔优化问题 36
7.3 对偶问题与 KKT 条件 36
7.3.1 为什么要转化为对偶问题? 36
7.3.2 拉格朗日对偶推导 36
7.3.3 KKT 条件 37
7.4 软间隔 SVM 37

7.4.1 为什么需要软间隔? 37
7.4.2 松弛变量与惩罚因子 37
7.5 核技巧 (Kernel Trick) 37
7.5.1 核心思想 37
7.5.2 常用核函数 38
7.5.3 核函数的合法性：Mercer 定理 38
7.6 SMO 算法简述 38
7.7 Hinge Loss 视角 38
7.8 关键代码片段 38
7.9 SVM vs 其他模型 39
7.10 深度面试题 39
7.11 参考文献与拓展 40

Chapter 1

Bagging

1.1 核心定义

中文定义：Bagging（套袋法），全称 **Bootstrap Aggregating**，是一种并行的集成学习方法。其核心思想是通过**自助采样（Bootstrap Sampling）**生成多个子训练集，在每个子集上训练一个基学习器，最后通过投票（分类）或平均（回归）的方式结合所有基学习器的预测结果。

英文定义：Bagging (Bootstrap Aggregating) is an ensemble learning technique that trains multiple base models independently on different subsets of the data created via bootstrap sampling, then combines their predictions through voting or averaging to reduce variance.

核心直觉：”三个臭皮匠，顶个诸葛亮”。Bagging 利用多个基学习器之间的差异性来相互抵消误差，主要针对的是**降低模型的方差（Variance）**。

1.2 核心原理：Bootstrap Sampling

1.2.1 什么是自助采样?

给定包含 N 个样本的数据集 D，自助采样的过程如下：每次随机从 D 中挑选一个样本，复制到子集 D_i 中，然后**将样本放回** D。重复 N 次，得到一个包含 N 个样本的子集 D_i。

1.2.2 包外估计 (Out-of-Bag, OOB)

在自助采样中，一个样本在 N 次采样中始终未被抽到的概率为：

$$\lim_{N\to\infty}\left(1-\frac{1}{N}\right)^N \approx \frac{1}{e} \approx 63.2\%$$

这意味着约 36.8% 的样本不会出现在某个子训练集 D_i 中。这些样本被称为**包外数据 (OOB data)**。**妙用**：我们可以直接利用这些 OOB 样本来作为验证集，对基学习器的泛化性能进行评估，而无需额外的交叉验证。

1.3 数学原理：为什么能降低方差?

假设我们有 n 个独立同分布（i.i.d.）的基学习器，每个学习器的预测方差为 σ^2。

- **基学习器之间完全独立**：集成的总方差为 $\frac{\sigma^2}{n}$。
- **基学习器之间存在相关性** ρ（现实情况）：集成的总方差为 $\rho\sigma^2 + \frac{1-\rho}{n}\sigma^2$。

可以看出：1. 集成的方差会随着基学习器数量 n 的增加而显著下降。2. 即使基学习器之间有正相关（因为它们来自同一个原始数据集），集成的方差仍小于单个学习器。这就是为什么 Bagging 特别适合**高方差模型**（如不加限制的决策树）。

1.4 典型代表：随机森林 (Random Forest)

随机森林是 Bagging 的最具代表性的变体。它在 Bagging 基础上增加了**特征随机选择**：

- **样本随机性**：自助采样（Bootstrap）。
- **特征随机性**：在决策树的每个节点分裂时，随机选择 k 个特征（通常 $k = \sqrt{d}$），然后从中选优。

目的：进一步降低基学习器之间的相关性 ρ，从而通过集成更有效地降低整体方差。

1.5 Bagging vs. Boosting

特性	**Bagging**	**Boosting**
执行顺序	并行 (Parallel)	串行 (Sequential)
核心手段	自助采样 (Bootstrap)	权重调整 (Residual fitting)
优化的目标	降低方差 (Variance reduction)	降低偏差 (Bias reduction)
基学习器	强学习器 (如深层树)	弱学习器 (如浅层树)
基学习器关系	相互独立	后者依赖前者

1.6 关键代码片段

```python
import numpy as np
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

# 加载数据
data = load_wine()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.3)

# 基础模型: 决策树
base_clf = DecisionTreeClassifier(max_depth=5)

# 初始化 Bagging
# n_estimators: 学习器数量
# oob_score: 启用包外估计
bagging = BaggingClassifier(
    estimator=base_clf,
    n_estimators=50,
    oob_score=True,
    random_state=42
)

bagging.fit(X_train, y_train)

print(f"OOB Score: {bagging.oob_score_:.4f}")
print(f"Test Accuracy: {bagging.score(X_test, y_test):.4f}")
```

1.7 深度面试题

问：Bagging 为什么要用强学习器，而 Boosting 喜欢用弱学习器?

答：Bagging 的主要目的是通过均值化降低方差。对于高方差的强学习器（如深度决策树），Bagging 能有效消除其各个模型因噪声产生的误差。反之，若用低方差的弱学习器，集成后方差变化不大，偏差却依然很高。Boosting 是串行修正偏差。弱学习器偏差

高但泛化力强，通过多次迭代拟合残差，能在保持低方差的同时让偏差迅速下降。

问：Random Forest 为什么要在分裂时随机选特征？

答：这是为了**解耦（De-correlation）**。如果所有树都用最重要的那几个特征作为根节点，那么得到的树往往极其相似。随机选择特征强制让不同的树学习数据的不同侧面，降低了模型间的相关性 ρ，从而强化了 Bagging 的方差削减效果。

1.8 参考文献与拓展

1. **经典论文**: Breiman, L. (1996). *Bagging Predictors.* Machine Learning.
2. **Random Forest**: Breiman, L. (2001). *Random Forests.* Machine Learning.
3. **教材**: 周志华《机器学习》第 8.2 节（Bagging 与随机森林）。

Chapter 2

Boosting

2.1 核心定义

中文定义：Boosting（提升法）是一种串行的集成学习方法。其核心思想是将多个弱学习器（Weak Learners）组合成一个强学习器（Strong Learner）。不同于 Bagging 的并行采样，Boosting 通过**顺序训练**基学习器，后一个学习器致力于修正前一个学习器的错误。

英文定义: Boosting is a sequential ensemble learning technique that combines several weak learners to form a strong learner. Each subsequent model attempts to correct the errors made by the previous models, typically by focusing on difficult-to-predict samples.

核心直觉：" 勤能补拙"。Boosting 利用基学习器之间的依赖性，通过不断迭代、加权，主要针对的是**降低模型的偏差（Bias）**。

2.2 加法模型与前向分布算法

Boosting 模型通常表示为基学习器的**加法模型（Additive Model）**:

$$H(x) = \sum_{t=1}^{T} \alpha_t h_t(x)$$

其中 $h_t(x)$ 是基学习器，α_t 是其权重。为了求解该模型，Boosting 采用**前向分布算法（Forward Stagewise Algorithm）**：每次只学习一个基学习器及其权重，逐步逼近最优损失。

$$f_t(x) = f_{t-1}(x) + \alpha_t h_t(x)$$

2.3 典型算法 1：AdaBoost (Adaptive Boosting)

2.3.1 核心机制

AdaBoost 主要通过**调整样本权重**来实现" 错误修正"：

1. **初始权重**：给所有样本相同的初始权重 $w_{1,i} = 1/N$。
2. **基学习算法**：在加权样本上训练 $h_t(x)$。
3. **错误率计算**：$\epsilon_t = \sum_{i:h_t(x_i)\neq y_i} w_{t,i}$。
4. **模型权重**：$\alpha_t = \frac{1}{2}\ln\frac{1-\epsilon_t}{\epsilon_t}$。
5. **样本权重更新**：$w_{t+1,i} = \frac{w_{t,i}}{Z_t}\exp(-\alpha_t y_i h_t(x_i))$，即误分类样本权重增加，正确分类样本权重减小。

2.3.2 损失函数视角

AdaBoost 等价于极小化**指数损失函数（Exponential Loss）**：

$$\min \sum_{i=1}^{N} \exp(-y_i f(x_i))$$

指数损失是 0-1 损失的一个连续可微的上界。

2.4 典型代表 2：GBDT (Gradient Boosting Decision Tree)

2.4.1 核心思想：拟合负梯度

对于一般的损失函数 $L(y, f(x))$，GBDT 利用**损失函数的负梯度**在当前模型的值作为残差的近似值：

$$r_{it} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]_{f(x)=f_{t-1}(x)}$$

直观理解：GBDT 实际上是在函数空间上执行**梯度下降**，每一棵新增的树都是沿着损失函数下降最快的方向迈出的一步。

2.5 典型代表 3：XGBoost (Extreme Gradient Boosting)

XGBoost 是 GBDT 的高性能实现。它不仅在工程上极致优化，在数学上也引入了二阶信息。

2.5.1 目标函数的严谨推导

Step 1: 二阶泰勒展开。在第 t 轮迭代中，目标函数为:

$$Obj^{(t)} = \sum_{i=1}^{n} L(y_i, f_{t-1}(x_i) + f_t(x_i)) + \Omega(f_t)$$

利用二阶泰勒展开 $\Delta L \approx g_i \Delta f + \frac{1}{2} h_i \Delta f^2$，其中 g_i, h_i 是损失函数对 f_{t-1} 的一阶和二阶导:

$$Obj^{(t)} \approx \sum_{i=1}^{n} [g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)] + \Omega(f_t)$$

Step 2: 树的参数化与正则项。令叶子节点集合为 $j \in \{1, \ldots, T\}$，样本集 $I_j = \{i | q(x_i) = j\}$。正则项 $\Omega(f_t) = \gamma T + \frac{1}{2}\lambda \sum w_j^2$。定义层级汇总 $G_j = \sum_{i \in I_j} g_i, H_j = \sum_{i \in I_j} h_i$:

$$Obj^{(t)} = \sum_{j=1}^{T} [G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2] + \gamma T$$

Step 3: 最优权重与结构分数。解得最优叶子权重 $w_j^* = -\frac{G_j}{H_j + \lambda}$，代入得最优目标函数:

$$Obj^* = -\frac{1}{2} \sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T$$

2.5.2 分裂收益 (Gain) 公式

节点分裂后的增益计算:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

2.6 典型代表 4: LightGBM (Light Gradient Boosting Machine)

2.6.1 GOSS (基于梯度的单侧采样)

为了在不牺牲太多精度的前提下减少计算量，GOSS 采用如下采样策略:

1. 将样本按梯度绝对值排序。
2. 保留前 $a \times 100\%$ 的大梯度样本（这些样本通常更难拟合）。
3. 从剩余样本中随机采样 $b \times 100\%$，并在计算信息增益时给这些样本权重放大 $\frac{1-a}{b}$ 倍。

2.6.2 EFB (互斥特征捆绑)

通过由于特征通常是稀疏且互斥的（即不同时为非零），EFB 将互斥特征捆绑成一个，从而降低特征维度，提高构建直方图的速度。

2.6.3 Leaf-wise vs Level-wise

- **Level-wise** (XGBoost): 按层生长，同一层节点同时分裂。稳健但有些节点增益很小，计算冗余。
- **Leaf-wise** (LightGBM): 每次选增益最大的节点分裂。效率极高，路径较深，容易过拟合（需依靠 `max_depth` 约束）。

2.7 Boosting 家族综合对比

特征	XGBoost	LightGBM	CatBoost
分裂准则	二阶泰勒展开 (Gain)	直方图算法	二进制对称树
生长策略	Level-wise	Leaf-wise	Level-wise
降维手段	列采样 (Column Subsampling)	GOSS + EFB	排序提升 (Ordered Boosting)
缺失值	自动学习默认方向	划分到特定桶	默认处理

2.8 关键代码片段

```python
import xgboost as xgb
import lightgbm as lgb
from sklearn.metrics import accuracy_score

# XGBoost 示例
dtrain = xgb.DMatrix(X_train, label=y_train)
params_xgb = {'objective': 'binary:logistic', 'max_depth': 4}
bst_xgb = xgb.train(params_xgb, dtrain, num_boost_round=50)

# LightGBM 示例
train_data = lgb.Dataset(X_train, label=y_train)
params_lgb = {'objective': 'binary', 'num_leaves': 31, 'learning_rate': 0.05}
bst_lgb = lgb.train(params_lgb, train_data, num_boost_round=100)
```

2.9 深度面试题

问：XGBoost 相比 GBDT 做了哪些改进？ 答：1. 二阶导：利用二阶导信息，收敛更快；2. 正则化：目标函数含叶子数和权重平方项，抗过拟合；3. 列采样：借鉴随机森林防止过拟合；4. 稀疏感知：自动处理缺失值；5. 并行优化：块存储支持特征并行计算。

2.10 参考文献

1. Chen, T., Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System.*

2. Ke, G., et al. (2017). *LightGBM: A Highly Efficient Gradient Boosting Decision Tree.*

3. 周志华《机器学习》第 8 章。

Chapter 3

Decision_tree

3.1 核心定义

中文定义：决策树是一种基于树形结构的监督学习算法。它通过递归地选择最优特征对数据集进行划分，从根节点到叶节点形成一系列 **if-then** 规则，最终实现分类或回归。

英文定义：A Decision Tree is a supervised learning algorithm that recursively partitions the feature space by selecting the most informative feature at each internal node, forming a tree of if-then rules for classification or regression.

核心优势：决策树是少数真正**可解释**的模型之一——人类可以直接阅读和理解其决策逻辑，在金融风控、医疗诊断等需要可解释性的领域不可替代。

3.2 核心原理：如何选择最优划分特征?

决策树构建的关键问题是：在每个节点上，选择哪个特征、以什么阈值进行划分?不同算法给出了不同的度量标准。

3.2.1 信息增益 (Information Gain) —ID3

信息熵衡量数据的不确定性：

$$H(D) = -\sum_{k=1}^{K} p_k \log_2 p_k$$

变量含义：

- D：当前数据集。
- K：类别总数。

- p_k：第 k 类样本在 D 中的占比。
- $H(D) \in [0, \log_2 K]$：熵越大，数据越”混乱”。

按特征 A 划分后的**条件熵**：

$$H(D|A) = \sum_{v=1}^{V} \frac{|D^v|}{|D|} H(D^v)$$

其中 D^v 是特征 A 取值为 v 的子集，V 是 A 的取值数。

信息增益：

$$\text{Gain}(D, A) = H(D) - H(D|A)$$

选择信息增益最大的特征进行划分。

ID3 算法的缺陷：信息增益偏向取值数目多的特征。例如，若存在一个”ID” 特征（每个样本唯一），它的信息增益最大，但毫无泛化能力。

3.2.2 信息增益率 (Gain Ratio) —C4.5

C4.5 引入**分裂信息量**对信息增益进行惩罚：

$$\text{SplitInfo}(D, A) = -\sum_{v=1}^{V} \frac{|D^v|}{|D|} \log_2 \frac{|D^v|}{|D|}$$

$$\text{GainRatio}(D, A) = \frac{\text{Gain}(D, A)}{\text{SplitInfo}(D, A)}$$

分裂信息量 SplitInfo 在特征取值多时会变大，从而自动惩罚” 取值过多” 的特征。

3.2.3 基尼指数 (Gini Index) —CART

基尼指数衡量从数据集中随机抽取两个样本、类别不一致的概率：

$$\text{Gini}(D) = 1 - \sum_{k=1}^{K} p_k^2$$

按特征 A 划分后：

$$\text{Gini}(D, A) = \sum_{v=1}^{V} \frac{|D^v|}{|D|} \text{Gini}(D^v)$$

选择使 $\text{Gini}(D, A)$ 最小的特征和划分点。CART 只进行**二叉划分**（将特征取值分为两组），每次产生恰好两个子节点。

算法	划分标准	树结构	核心特点
ID3	信息增益	多叉树	只能处理离散特征，偏好多值特征。
C4.5	信息增益率	多叉树	支持连续特征，修正了 ID3 的偏好问题。
CART	基尼指数	二叉树	既可分类也可回归，sklearn 默认实现。

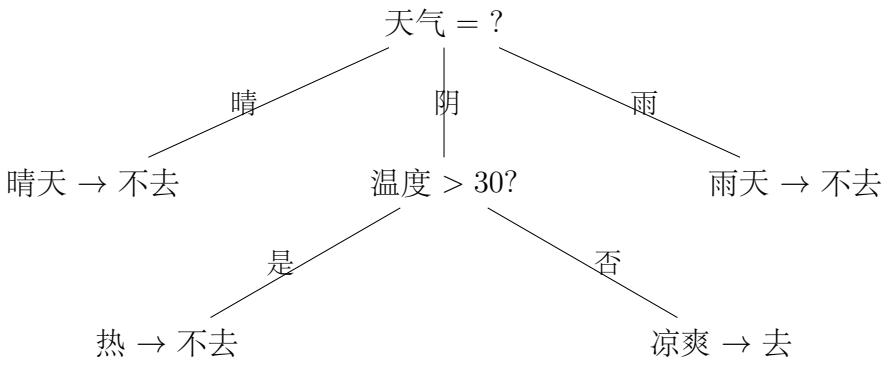

图 3.1: 决策树示意图：根据天气和温度决定是否出行

3.3 三大经典算法对比

3.4 决策树示意图

3.5 剪枝 (Pruning)

决策树如果不加限制地生长，会完美拟合训练数据（每个叶子节点只包含一个样本），导致严重**过拟合**。剪枝是决策树最重要的正则化手段。

3.5.1 预剪枝 (Pre-pruning)

在构建树的过程中提前停止分裂。常用策略：

- 设定最大深度 (`max_depth`)。
- 设定叶子节点最小样本数 (`min_samples_leaf`)。
- 设定分裂所需最小样本数 (`min_samples_split`)。
- 设定最小信息增益阈值。

优点：训练速度快。**缺点**：可能过早停止分裂，导致欠拟合。

3.5.2 后剪枝 (Post-pruning)

先构建完整的决策树，再自底向上地评估每个子树：如果去掉子树后在验证集上的性能不下降（或仅微弱下降），则将该子树替换为叶子节点。**优点**：通常能获得更好的泛化性能。**缺点**：需要额外的验证集，计算代价较高。

3.6 CART 回归树

CART 不仅能分类，还能回归。回归树在每个叶子节点输出该区域所有训练样本目标值的**均值**，划分标准改为最小化**均方误差 (MSE)**：

$$\min_{A,s}\left[\sum_{x_i\in D_1(A,s)}(y_i-c_1)^2+\sum_{x_i\in D_2(A,s)}(y_i-c_2)^2\right]$$

其中 c_1, c_2 分别是左右子树的均值。

3.7 集成学习扩展

单棵决策树的能力有限，但通过集成方法可以显著提升性能：

- **Bagging (Random Forest)**：多棵树并行训练，投票/平均，降低方差。
- **Boosting (GBDT, XGBoost, LightGBM)**：多棵树串行训练，每棵修正前一棵的残差，降低偏差。
- 集成学习是当前表格数据竞赛中的**绝对统治者**。

3.8 关键代码片段

```python
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# 加载数据
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.3)

# 构建 CART 决策树
tree = DecisionTreeClassifier(
```

```
    criterion='gini',        # 基尼指数
    max_depth=3,              # 预剪枝：最大深度
    min_samples_leaf=5        # 预剪枝：叶子最小样本数
)
tree.fit(X_train, y_train)
print(f"Accuracy: {tree.score(X_test, y_test):.4f}")

# 打印树的文本表示
print(export_text(tree, feature_names=load_iris().feature_names))
```

3.9 深度面试题

问：为什么 Random Forest 通常比单棵决策树好？

答：单棵决策树是高方差模型（不稳定，数据微小变化会导致完全不同的树）。Random Forest 通过 1. Bagging（有放回采样）降低方差；2. 特征随机选择进一步降低树之间的相关性。根据方差分解公式：$\mathrm{Var}(\bar{X}) = \frac{\rho\sigma^2+(1-\rho)\sigma^2/n}{1}$，当树之间相关性 ρ 降低时，集成的方差大幅下降。

问：决策树处理缺失值的策略是什么？

答：C4.5 的方案是：在计算信息增益时，对缺失值样本按比例分配到各个子节点（权重按非缺失样本的分布比例确定）。CART 使用代理变量（Surrogate Split）：选择一个与原始最优特征最相似的替代特征来处理缺失值。

3.10 参考文献与拓展

1. **ID3**: Quinlan, J. R. (1986). *Induction of decision trees.* Machine Learning.
2. **C4.5**: Quinlan, J. R. (1993). *C4.5: Programs for Machine Learning.* Morgan Kaufmann.
3. **CART**: Breiman, L., et al. (1984). *Classification and Regression Trees.* Wadsworth.
4. **教材**: 周志华《机器学习》第 4 章（决策树）。

Chapter 4

Lda

4.1 核心定义

中文定义：线性判别分析（Linear Discriminant Analysis, LDA），也称为 Fisher 线性判别（Fisher's Linear Discriminant, FLD），是一种经典的监督学习投影方法。其核心思想是将高维样本投影到低维空间，使得投影后的数据在新的维度上**类内方差最小、类间方差最大**。

英文定义：Linear Discriminant Analysis (LDA) is a supervised dimensionality reduction and classification technique that projects data into a lower-dimensional space while maximizing the ratio of between-class variance to within-class variance.

核心直觉：想象两坨散布在二维平面上的点。LDA 的目标是找一条直线，把这些点投射上去后，同类别的点靠得尽可能近（类内紧凑），不同类别的中心点离得尽可能远（类间分离）。

4.2 数学推导：Fisher 判别准则

4.2.1 类内散度矩阵 (Within-class Scatter Matrix)

设数据集 $D = \{(x_1, y_1), \ldots, (x_N, y_N)\}$，第 $i \in \{0, 1\}$ 类的均值向量为 μ_i，样本集合为 X_i。类内散度矩阵 S_w 定义为：

$$S_w = \Sigma_0 + \Sigma_1$$

其中 $\Sigma_i = \sum_{x \in X_i} (x - \mu_i)(x - \mu_i)^\top$。它衡量了每一类内部的数据发散程度。

4.2.2 类间散度矩阵 (Between-class Scatter Matrix)

类间散度矩阵 S_b 定义为：

$$S_b = (\mu_0 - \mu_1)(\mu_0 - \mu_1)^\top$$

它衡量了不同类中心之间的距离。

4.2.3 最优化目标

设投影方向为向量 w。投影后的类间方差为 $w^\top S_b w$，类内方差为 $w^\top S_w w$。Fisher 判别准则的目标是最大化广义瑞利商：

$$J(w) = \frac{w^\top S_b w}{w^\top S_w w}$$

4.2.4 解的求取

最大化 $J(w)$ 等价于在约束 $w^\top S_w w = 1$ 下最大化 $w^\top S_b w$。利用拉格朗日乘子法，解满足：

$$S_b w = \lambda S_w w$$

对于二分类问题，由于 $S_b w$ 的方向始终在 $(\mu_0 - \mu_1)$ 上，我们可以直接得到最优投影方向：

$$w = S_w^{-1}(\mu_0 - \mu_1)$$

注意：实际应用中常对 S_w 进行正则化（如 $S_w + \epsilon I$）以保证其可逆性。

4.3 多分类扩展

对于 $N > 2$ 的分类任务，LDA 将数据投影到 $M - 1$ 维空间（M 为类别数）。

- **全局均值**：$\mu = \frac{1}{N}\sum n_i \mu_i$。
- **多类类间散度**：$S_b = \sum_{i=1}^{M} n_i(\mu_i - \mu)(\mu_i - \mu)^\top$。
- **多类类内散度**：$S_w = \sum_{i=1}^{M} S_{w,i}$。

此时需要求解矩阵 $S_w^{-1} S_b$ 的最大特征值对应的特征向量。

4.4 LDA vs. PCA

这是面试中最常问的对比。

4.5 局限性

1. **线性假设**：LDA 假设数据分布是高斯分布，且各类的协方差矩阵相同。如果不满足（如非线性分布），效果可能不如核 LDA (Kernel LDA)。

特性	PCA (主成分分析)	LDA (线性判别分析)
监督类型	无监督 (Unsupervised)	有监督 (Supervised)
核心目标	最大化投影后的总方差 (保持信息)	最大化类间/类内方差比 (分类)
降维上限	特征维度 d	类别数减一 $M-1$
适用场景	数据压缩、噪声过滤	预处理以进行分类

2. **降维限制**：降维后的维度最多为 $M-1$，这在类别数较少时限制了特征的保留。

3. **对异常值敏感**：均值的计算容易受到离群点的影响。

4.6 关键代码片段

```python
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.datasets import load_iris

# 加载数据 (Iris 3类别)
data = load_iris()
X, y = data.data, data.target

# 初始化并训练 LDA
# n_components 最多为 类别数-1 = 2
lda = LinearDiscriminantAnalysis(n_components=2)
X_lda = lda.fit_transform(X, y)

print(f"原始维度: {X.shape[1]}")
print(f"降维后维度: {X_lda.shape[1]}")
print(f"可解释方差比: {lda.explained_variance_ratio_}")
```

4.7 深度面试题

问：LDA 降维后的维度上限为什么是 $M-1$?

答：因为类间散度矩阵 S_b 是由 M 个均值向量 $(\mu_i-\mu)$ 的外积之和构成的。由于这 M 个向量的线性组合为 0（即 $\sum n_i(\mu_i-\mu)=0$），所以这组向量中最多只有 $M-1$ 个是线性无关的。因此，S_b 的秩（Rank）最大为 $M-1$。矩阵 $S_w^{-1}S_b$ 的非零特征值最多也就只有 $M-1$ 个。

问：LDA 一定比 PCA 更适合分类任务吗?

答：不一定。虽然 LDA 利用了标签信息，但它假设类内分布服从高斯分布且协方差相

同。如果数据的方差主要由非相关特征引起，或者类别是高度非线性的（比如环形分布），PCA 保持最大方差或核 PCA 往往能提供比 LDA 更好的特征。

4.8 参考文献与拓展

1. **经典论文**: Fisher, R. A. (1936). *The use of multiple measurements in taxonomic problems.* Annals of Eugenics.

2. **教材**: 周志华《机器学习》第 3.4 节（线性判别分析）。

3. **理论进阶**: *Elements of Statistical Learning*, Chapter 4.3.

Chapter 5

Logistic_regression

5.1 核心定义

中文定义：逻辑回归（Logistic Regression）是一种广义线性模型，虽然名字含有”回归”，但它主要用于**分类**任务。其核心思想是通过 Sigmoid 函数将线性组合的输出映射到 $(0,1)$ 区间，表示样本属于某一类的概率。

英文定义：Logistic Regression is a generalized linear model used primarily for classification. It models the probability of the positive class by applying the logistic (sigmoid) function to a linear combination of input features.

历史地位：逻辑回归是机器学习中最经典、最基础的分类算法之一，也是深度学习中神经元和交叉熵损失函数的直接来源。理解逻辑回归是理解神经网络的重要前提。

5.2 数学建模：从线性回归到逻辑回归

5.2.1 为什么不能直接用线性回归做分类?

线性回归的输出 $\hat{y} = w^\top x + b$ 范围是 $(-\infty, +\infty)$，无法直接解释为概率。若强行用阈值划分，当数据分布变化时，决策边界会剧烈偏移，鲁棒性极差。

5.2.2 对数几率视角：Sigmoid 从何而来?

逻辑回归的名字来源于 **Logit**（对数几率），其推导过程比直接” 选一个 Sigmoid” 更加自然和深刻。

Step 1: 定义几率 (Odds)。对于一个事件发生的概率 p，**几率**定义为该事件发生与不发生的概率之比：

$$\text{Odds} = \frac{p}{1-p}$$

几率的范围是 $(0, +\infty)$。例如，$p = 0.8$ 时 $\text{Odds} = 4$，表示” 发生的可能性是不发生的 4 倍”。

Step 2: 取对数得到 Logit。几率的范围是 $(0, +\infty)$，仍然不对称。对其取自然对数：

$$\text{logit}(p) = \ln \frac{p}{1-p}$$

此时 logit 的值域变为 $(-\infty, +\infty)$，与线性模型的输出空间**完美匹配**！

Step 3: 令 logit 等于线性模型。这是逻辑回归最核心的建模假设——我们假设**对数几率**是自变量的线性函数：

$$\ln \frac{p}{1-p} = w^\top x + b$$

这个假设非常自然：线性模型擅长建模 $(-\infty, +\infty)$ 范围的量，而 logit 恰好就是这样的量。

Step 4: 反解出 Sigmoid。从上式解出 p：

$$\frac{p}{1-p} = e^{w^\top x + b} \tag{5.1}$$

$$p = e^{w^\top x + b}(1-p) \tag{5.2}$$

$$p(1 + e^{w^\top x + b}) = e^{w^\top x + b} \tag{5.3}$$

$$p = \frac{e^{w^\top x + b}}{1 + e^{w^\top x + b}} = \frac{1}{1 + e^{-(w^\top x + b)}} = \sigma(w^\top x + b) \tag{5.4}$$

Sigmoid 函数就这样自然地”涌现”了——它不是人为选择的，而是从”对数几率是线性函数”这一假设中数学推导出来的。

5.2.3 Sigmoid 函数的关键性质

逻辑回归引入 Sigmoid（Logistic）函数：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

变量含义：

- $z = w^\top x + b$：线性组合的输出，称为 **logit**。
- $w \in \mathbb{R}^d$：权重向量，d 为特征维度。
- $b \in \mathbb{R}$：偏置项。
- $\sigma(z) \in (0, 1)$：输出的概率值。

关键性质：

1. $\sigma(0) = 0.5$（决策边界）。
2. $\sigma(-z) = 1 - \sigma(z)$（对称性）。
3. 导数形式优雅：$\sigma'(z) = \sigma(z)(1 - \sigma(z))$，这使得梯度计算非常简洁。

5.2.4 概率模型

逻辑回归假设：

$$P(y=1|x)=\sigma(w^\top x+b),\quad P(y=0|x)=1-\sigma(w^\top x+b)$$

可以统一写为：

$$P(y|x)=\hat{y}^y(1-\hat{y})^{1-y},\quad \hat{y}=\sigma(w^\top x+b)$$

5.3 损失函数推导：为什么用交叉熵?

5.3.1 极大似然估计 (MLE)

给定 N 个独立同分布的训练样本 $\{(x_i,y_i)\}$，似然函数为：

$$L(w,b)=\prod_{i=1}^{N}\hat{y}_i^{y_i}(1-\hat{y}_i)^{1-y_i}$$

取负对数得到**交叉熵损失函数**（Binary Cross-Entropy, BCE）：

$$\mathcal{L}(w,b)=-\frac{1}{N}\sum_{i=1}^{N}[y_i\log\hat{y}_i+(1-y_i)\log(1-\hat{y}_i)]$$

为什么不用 MSE?

- MSE 与 Sigmoid 结合后损失函数是**非凸**的，存在大量局部最优。
- 交叉熵损失函数关于 w 是**凸函数**，保证全局最优解唯一。
- 交叉熵的梯度形式更简洁，训练效率更高。

5.3.2 梯度推导

对权重 w_j 求偏导：

$$\frac{\partial\mathcal{L}}{\partial w_j}=\frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i-y_i)\cdot x_{ij}$$

推导要点：

1. 利用 $\sigma'(z)=\sigma(z)(1-\sigma(z))$。
2. 对数求导后与 Sigmoid 导数相消，最终形式极其简洁。
3. 更新规则：$w_j\leftarrow w_j-\alpha\cdot\frac{\partial\mathcal{L}}{\partial w_j}$，其中 α 是学习率。

这个梯度形式与线性回归的梯度形式相同（$\hat{y}_i-y_i$ 乘以输入），是逻辑回归数学优雅性的体现。

5.4 决策边界

逻辑回归的决策边界由 $w^\top x + b = 0$ 定义，这是一个**线性超平面**。

- 在二维空间中，决策边界是一条直线。
- 在高维空间中，决策边界是一个超平面。
- 这意味着逻辑回归本质上是一个**线性分类器**，无法处理非线性可分的数据（如 XOR 问题）。

解决非线性：可以通过手动构造多项式特征或使用核技巧（Kernel Trick）来扩展。

5.5 正则化

为防止过拟合，通常在损失函数中加入正则项：

- **L2 正则化**（Ridge）：$\mathcal{L}_{reg} = \mathcal{L} + \frac{\lambda}{2}\|w\|_2^2$，使权重趋向较小值。
- **L1 正则化**（Lasso）：$\mathcal{L}_{reg} = \mathcal{L} + \lambda\|w\|_1$，产生稀疏权重，可用于特征选择。
- **Elastic Net**：L1 + L2 的组合。

5.6 多分类扩展：Softmax 回归

当类别数 $K > 2$ 时，逻辑回归可扩展为 **Softmax 回归**（多项逻辑回归）：

$$P(y = k|x) = \frac{e^{w_k^\top x + b_k}}{\sum_{j=1}^{K} e^{w_j^\top x + b_j}}$$

变量含义：

- K：总类别数。
- w_k, b_k：第 k 类的权重向量和偏置。
- 分母是归一化常数，确保所有类别概率之和为 1。

对应的损失函数为**多类交叉熵**：

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik} \log P(y_i = k|x_i)$$

5.7 关键代码片段

```python
import numpy as np

class LogisticRegression:
    def __init__(self, lr=0.01, n_iters=1000):
        self.lr = lr
        self.n_iters = n_iters
        self.w = None
        self.b = None

    def _sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.w = np.zeros(n_features)
        self.b = 0
        for _ in range(self.n_iters):
            z = np.dot(X, self.w) + self.b
            y_hat = self._sigmoid(z)
            # 梯度 = (1/N) * X^T * (y_hat - y)
            dw = (1 / n_samples) * np.dot(X.T, (y_hat - y))
            db = (1 / n_samples) * np.sum(y_hat - y)
            self.w -= self.lr * dw
            self.b -= self.lr * db

    def predict(self, X):
        z = np.dot(X, self.w) + self.b
        return (self._sigmoid(z) >= 0.5).astype(int)
```

5.8 逻辑回归 vs 其他模型

5.9 深度面试题

问：逻辑回归的损失函数为什么是凸的?

答：BCE 损失函数的 Hessian 矩阵为 $H = \frac{1}{N}X^\top DX$，其中 $D = \mathrm{diag}(\hat{y}_i(1-\hat{y}_i))$ 是一个对角矩阵且对角元素恒正。因此 H 是半正定矩阵，损失函数是凸函数。

对比维度	分析
vs 线性回归	线性回归解决回归问题，逻辑回归解决分类问题。逻辑回归多了一个 Sigmoid 激活。
vs SVM	SVM 关注最大化间隔（几何直觉），逻辑回归关注最大化似然（概率直觉）。SVM 不直接输出概率。
vs 决策树	决策树能处理非线性，但容易过拟合。逻辑回归是线性模型，更稳健但表达力有限。
vs 神经网络	单个神经元 + Sigmoid 激活 = 逻辑回归。神经网络是多层逻辑回归的堆叠。

问：逻辑回归能否处理多重共线性？

答：逻辑回归对多重共线性敏感。当特征高度相关时，权重估计的方差会变大，模型不稳定。解决方法：1. L2 正则化；2. 主成分分析（PCA）降维；3. 特征选择。

5.10 参考文献与拓展

1. **经典教材**: Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*, Chapter 4.

2. **统计视角**: Hastie, T., et al. (2009). *The Elements of Statistical Learning*, Chapter 4.3.

3. **工程实践**: `sklearn.linear_model.LogisticRegression` 官方文档。

Chapter 6

Pca

6.1 核心定义

中文定义：主成分分析（Principal Component Analysis, PCA）是一种最常用的无监督线性降维算法。其通过线性变换将原始数据投影到一组正交的新坐标系中，使得投影后的数据在第一个坐标轴（第一主成分）上的方差最大，第二个坐标轴（第二主成分）上的方差次之，以此类推。

英文定义：Principal Component Analysis (PCA) is an unsupervised linear dimensionality reduction technique that transforms data into a new orthogonal coordinate system such that the greatest variance by some scalar projection of the data comes to lie on the first coordinate (first principal component), the second greatest variance on the second coordinate, and so on.

核心直觉：找一个投影方向，使得投影后的点" 散得最开"，从而尽可能多地保留原始数据的分布信息。

6.2 数学推导：最大方差视角

6.2.1 问题建模

设定原始数据集为 $X = [x_1, x_2, \ldots, x_N] \in \mathbb{R}^{d\times N}$，且数据已做**零均值化**处理（即 $\sum x_i = 0$）。我们要找一个投影向量 w（满足 $\|w\| = 1$），使得样本点投射到 w 后的方差最大。投影后的样本点为 $w^\top x_i$，其方差为：

$$\mathrm{Var} = \frac{1}{N}\sum_{i=1}^{N}(w^\top x_i)^2 = w^\top \left(\frac{1}{N}\sum_{i=1}^{N} x_i x_i^\top\right) w$$

令协方差矩阵 $C = \frac{1}{N}XX^\top$，目标变为最小化：

$$\max_w w^\top C w \quad \text{s.t.} \quad w^\top w = 1$$

6.2.2 拉格朗日乘子法求解

引入拉格朗日乘子 λ:

$$L(w, \lambda) = w^\top C w - \lambda(w^\top w - 1)$$

对 w 求导并令其为 0:

$$\frac{\partial L}{\partial w} = 2Cw - 2\lambda w = 0 \implies Cw = \lambda w$$

结论：投影方向 w 必须是协方差矩阵 C 的**特征向量**。代入原式发现方差就是特征值 λ。因此，最大方差对应的方向就是最大特征值对应的特征向量。

6.3 另一种视角：最小重构误差

PCA 也可以理解为寻找一个超平面对原始空间进行投影，使得投影后的样本点恢复到原始空间时的**均方误差最小**。

$$\min \sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2$$

推导结果与最大方差视角完全等价。

6.4 算法步骤

1. **去中心化**：对所有样本进行 $x_i \leftarrow x_i - \mu$，其中 μ 为均值。
2. **计算协方差矩阵**：$C = \frac{1}{N} X X^\top$。
3. **特征值分解**：计算 C 的特征值和特征向量。
4. **选择主成分**:将特征值从大到小排序,选择前 k 个特征值对应的特征向量 $w_1, \ldots, w_k$。
5. **投影降维**：$X' = W^\top X$。

6.5 SVD 实现的优势

在实际工程中，通常不直接对协方差矩阵进行特征分解，而是对原始矩阵 X 进行**奇异值分解 (SVD)**:

$$X = U \Sigma V^\top$$

协方差矩阵 $XX^\top = U\Sigma\Sigma^\top U^\top$。可以看出，特征分解中的特征向量就是 SVD 中的左奇异向量 U。**优点**:

1. SVD 不需要显式计算巨大的协方差矩阵（保存 $d \times d$ 矩阵可能溢出内存）。
2. SVD 的数值计算比特征分解更稳定（精度更高）。

6.6 PCA vs. LDA

特性	PCA	LDA
是否监督	无监督	有监督 (利用标签)
核心目标	最大化全局方差 (保持信息)	最大化类间方差/类内方差比 (分类)
降维限制	无，$1 \sim d$ 均可	类别数减一 $M-1$

6.7 关键代码片段

```python
import numpy as np
from sklearn.decomposition import PCA
from sklearn.datasets import load_digits

# 加载手写数字数据集 (64维)
X, _ = load_digits(return_X_y=True)

# 初始化 PCA
pca = PCA(n_components=0.95) # 保留 95% 的方差信息
X_reduced = pca.fit_transform(X)

print(f"原始特征数: {X.shape[1]}")
print(f"降维后特征数: {X_reduced.shape[1]}")
print(f"各个主成分解释的方差比: {pca.explained_variance_ratio_}")
```

6.8 深度面试题

问：PCA 前为什么一定要做中心化（Normalization/Scaling)?

答：1. **均值归零**：PCA 推导假设协方差矩阵定义在均值为 0 的基础上，否则第一个主成分可能会指向均值方向而非方差方向；2. **尺度缩放**：PCA 对特征量级敏感。如果某个特征单位是”公里”，另一个是”厘米”，前者方差会远大，导致 PCA 误认为其携带更多信息。

问：如何选择主成分个数 k?

答：通常使用”方差累计贡献率”：

$$\eta = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{j=1}^{d} \lambda_j} \geq t$$

常用的阈值 t 设为 95% 或 99%。

6.9 参考文献

1. Pearson, K. (1901). *On lines and planes of closest fit to systems of points in space.*

2. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*, Chapter 12.

3. 同济大学数学系《线性代数》。

Chapter 7

Svm

7.1 核心定义

中文定义：支持向量机（Support Vector Machine, SVM）是一种基于**最大间隔**原理的监督学习算法。其核心思想是在特征空间中找到一个最优超平面，使得该超平面到最近的训练样本（支持向量）的距离最大化。

英文定义：Support Vector Machine (SVM) is a supervised learning algorithm that finds the optimal hyperplane which maximizes the margin between the closest data points (support vectors) of different classes.

历史地位：SVM 由 Vapnik 等人在 1990 年代提出，在深度学习兴起之前，SVM 是机器学习领域最强大的分类算法之一，广泛应用于文本分类、图像识别和生物信息学。其背后的统计学习理论（VC 维、结构风险最小化）至今仍是机器学习理论的基石。

7.2 线性可分 SVM：硬间隔

7.2.1 几何直觉：为什么要最大化间隔?

给定线性可分的二分类数据集，能将数据分开的超平面有无穷多个。SVM 的核心哲学是：选择**间隔最大**的那个超平面，因为它对未见数据的泛化能力最强（鲁棒性最好）。

7.2.2 数学建模

超平面定义为 $w^\top x + b = 0$，其中：

- $w \in \mathbb{R}^d$：法向量，决定超平面的方向。
- $b \in \mathbb{R}$：偏置，决定超平面的位移。
- 点 x_i 到超平面的**几何距离**：$\gamma_i = \frac{|w^\top x_i + b|}{\|w\|}$。

7.2.3 最大间隔优化问题

设标签 $y_i \in \{-1, +1\}$，SVM 的原始优化问题为：

$$\max_{w,b} \frac{2}{\|w\|} \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1, \quad \forall i$$

等价于（取倒数并平方）：

$$\min_{w,b} \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^\top x_i + b) \geq 1, \quad \forall i$$

变量含义：

- $\frac{2}{\|w\|}$：两类支持向量之间的**间隔宽度**（Margin）。
- $y_i(w^\top x_i + b) \geq 1$：约束条件，确保所有样本被正确分类且距离超平面至少为 $\frac{1}{\|w\|}$。
- **支持向量**：满足 $y_i(w^\top x_i + b) = 1$ 的样本点，它们恰好落在间隔的边界上。

7.3 对偶问题与 KKT 条件

7.3.1 为什么要转化为对偶问题?

1. 引入拉格朗日乘子后，问题从 d 维（特征维度）转化为 N 维（样本数量），在高维特征空间中更高效。
2. 对偶形式中目标函数只涉及 $x_i^\top x_j$（内积），为**核技巧**的使用奠定了基础。

7.3.2 拉格朗日对偶推导

引入拉格朗日乘子 $\alpha_i \geq 0$，构造拉格朗日函数：

$$L(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{N} \alpha_i \left[y_i(w^\top x_i + b) - 1\right]$$

分别对 w 和 b 求偏导并令其为零：

$$\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^{N} \alpha_i y_i x_i \tag{7.1}$$

$$\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_{i=1}^{N} \alpha_i y_i = 0 \tag{7.2}$$

代入 L 得到**对偶问题**：

$$\max_{\alpha} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j x_i^\top x_j$$

$$\text{s.t.} \quad \alpha_i \geq 0, \quad \sum_{i=1}^{N} \alpha_i y_i = 0$$

7.3.3 KKT 条件

对于最优解，必须满足 KKT 互补松弛条件：

$$\alpha_i \left[y_i(w^\top x_i + b) - 1 \right] = 0$$

关键洞察：若 $\alpha_i > 0$, 则 $y_i(w^\top x_i + b) = 1$, 该样本恰好是**支持向量**。大多数样本的 $\alpha_i = 0$，不影响决策边界——这就是 SVM 的**稀疏性**。

7.4 软间隔 SVM

7.4.1 为什么需要软间隔?

现实数据往往线性不完全可分，或者存在噪声点。硬间隔 SVM 对异常值极度敏感，一个噪声点就能彻底改变决策边界。

7.4.2 松弛变量与惩罚因子

引入松弛变量 $\xi_i \geq 0$ 允许部分样本违反间隔约束：

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N} \xi_i$$

$$\text{s.t.} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

变量含义：

- ξ_i：第 i 个样本允许的违反量。$\xi_i = 0$ 表示正确分类且在间隔外；$0 < \xi_i < 1$ 表示在间隔内但正确分类；$\xi_i \geq 1$ 表示被错误分类。
- $C > 0$：**惩罚因子**，控制" 间隔最大化" 与" 分类错误最小化" 的权衡。C 越大越不容忍错误，C 越小越追求宽间隔。

7.5 核技巧 (Kernel Trick)

7.5.1 核心思想

当数据在原始空间非线性可分时，将数据映射到**高维特征空间** $\phi(x)$，在高维空间中寻找线性超平面。

关键问题：高维映射 $\phi(x)$ 的计算代价极高（甚至无穷维）。

核技巧的巧妙之处：由于对偶问题中只涉及内积 $x_i^\top x_j$，因此只需要定义一个**核函数** $K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$，而**不需要显式计算** $\phi(x)$。

核函数	公式	特点
线性核	$K(x_i, x_j) = x_i^\top x_j$	不做映射，等价于线性 SVM。
多项式核	$K(x_i, x_j) = (x_i^\top x_j + c)^d$	映射到有限维空间，d 控制复杂度。
RBF (高斯核)	$K(x_i, x_j) = e^{-\gamma\|x_i - x_j\|^2}$	映射到无穷维空间，最常用。γ 越大，决策边界越复杂。
Sigmoid 核	$K(x_i, x_j) = \tanh(\beta x_i^\top x_j + \theta)$	类似神经网络，不总满足 Mercer 条件。

7.5.2 常用核函数

7.5.3 核函数的合法性：Mercer 定理

一个函数 $K(x, z)$ 能作为合法的核函数，当且仅当对于任意有限样本集，由 $K(x_i, x_j)$ 构成的 **Gram 矩阵**是半正定的。

7.6 SMO 算法简述

序列最小优化（Sequential Minimal Optimization, SMO）是 SVM 最常用的求解算法，由 Platt (1998) 提出。其核心思想是每次只选取两个 α_i, α_j 进行优化（因为约束 $\sum \alpha_i y_i = 0$ 要求至少同时更新两个变量），将二次规划问题分解为大量的解析可解的子问题。

7.7 Hinge Loss 视角

SVM 也可以从损失函数角度理解。软间隔 SVM 等价于最小化：

$$\min_{w,b} \sum_{i=1}^{N} \max(0, 1 - y_i(w^\top x_i + b)) + \frac{\lambda}{2}\|w\|^2$$

其中 $\max(0, 1 - y_i f(x_i))$ 就是 **Hinge Loss**，它对正确分类且间隔充足的样本给予零损失，对违反间隔的样本给予线性惩罚。与之对比：

- **逻辑回归**使用 Log Loss：$\log(1 + e^{-yf(x)})$，对所有样本都有非零损失。
- **SVM** 使用 Hinge Loss：只关注间隔附近的样本（支持向量），这正是 SVM 稀疏性的来源。

7.8 关键代码片段

```
from sklearn.svm import SVC
from sklearn.datasets import make_moons
```

```python
from sklearn.model_selection import train_test_split

# 生成非线性可分数据
X, y = make_moons(n_samples=200, noise=0.15, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size
    =0.3)

# RBF 核 SVM
svm = SVC(kernel='rbf', C=1.0, gamma='scale')
svm.fit(X_train, y_train)

print(f"Support Vectors: {svm.n_support_}")  # 支持向量数量
print(f"Accuracy: {svm.score(X_test, y_test):.4f}")
```

7.9 SVM vs 其他模型

对比维度	分析
vs 逻辑回归	LR 最大化似然（概率视角），SVM 最大化间隔（几何视角）。SVM 不直接输出概率。
vs 决策树	决策树可解释性强但易过拟合，SVM 泛化能力强但可解释性差。
vs 神经网络	神经网络可自动学习特征，SVM 依赖核函数设计。在大数据上神经网络更优，小数据上 SVM 更稳健。

7.10 深度面试题

问：SVM 为什么对数据缩放敏感？

答：SVM 的目标是最大化间隔 $\frac{2}{\|w\|}$，间隔的计算依赖于特征的尺度。如果某个特征的数值范围远大于其他特征，该特征会主导距离计算，导致超平面偏向该特征的方向。因此使用 SVM 前通常需要对数据进行标准化。

问：RBF 核的 γ 参数如何影响模型？

答：γ 控制单个样本的影响范围。γ 越大，每个样本的影响范围越小，决策边界越复杂（容易过拟合）；γ 越小，影响范围越大，决策边界越平滑（可能欠拟合）。可以直觉理解为：γ 大相当于”近视”，只看最近的点。

7.11 参考文献与拓展

1. **经典论文**: Vapnik, V. (1995). *The Nature of Statistical Learning Theory.* Springer.
2. **SMO 算法**: Platt, J. (1998). *Sequential Minimal Optimization.* Microsoft Research Technical Report.
3. **教材**: 周志华《机器学习》第 6 章（支持向量机）。
4. **工程实践**: `sklearn.svm.SVC` 官方文档。